EE-5102/CS-6304 - Advanced Topics in Machine Learning – Fall 2026
Programming Assignment 3 – *Release Date: 23 October 2026*

Instructions

- This is the third graded programming assignment of the course.
- **Individual submission is mandatory.** Your code, experiments, results, analysis, and report must be your own. You may not collaborate with other students or share assignment work.
- The assignment is due on **06 November 2026 via LMS**.
- All submitted code must be available in a public GitHub repository.
- Submit an **8-page NeurIPS-style PDF report** presenting your experiments, results, and analysis. References do not count toward the page limit. Include the link to your GitHub repository at the end of the abstract. Further report-writing and submission requirements are provided in the Report and Submission Guidelines section.
- **AI Usage Policy:** You may not use generative AI to write any part of your PDF report. The report's language, interpretation, and analysis must be entirely your own.
- **Coding Assistance:** You may use public libraries, built-in functions, open-source implementations, and coding assistance from an LLM. However, you are responsible for every submitted line of code and must understand how it works. Clearly attribute materially reused external code in your README.
- You are highly encouraged to read through each task before starting it.
- Fix random seeds where practical, record all reported hyperparameters, and preserve the configurations used to produce your results. Extensive hyperparameter tuning is not expected unless a task explicitly requires it.

Overview & Motivation

Pretrained language and vision models provide useful starting points for new applications, but adapting and using them raises several questions. How does training on a new domain affect existing capabilities? How can a language model learn to use visual evidence? When does spending more computation at inference improve an answer? This assignment investigates these questions through three sets of experiments.

Task 1: Large Language Model Fine-Tuning. You will fine-tune a language model on mathematical solutions and measure both mathematical gains and changes on other tasks. Comparing full fine-tuning, starting-point regularization, low-rank adapters, and orthogonal protection will let you examine how restrictions on parameter updates affect adaptation, retention, and resource use.

Task 2: Vision-Language Models and Visual Grounding. You will connect a pretrained vision encoder to a language model and train the system through caption alignment and visual question answering. Complementary image pairs and image controls will help you investigate whether its answers depend on useful visual evidence or language priors.

Task 3: Test-Time Compute. You will use supplied LLM and VLM rollouts to study repeated sampling, confidence-based selection, and answer consensus. A small live-generation experiment compares independent alternatives with longer responses and sequential refinement. You will also implement adaptive sampling to investigate when additional candidates justify their computational cost.

Use controlled comparisons and informative failure cases to explain the observed results. Evaluate each method against its intended purpose and measured cost, and distinguish evidence about model behavior from possible explanations for it.

1 Large Language Model Fine-Tuning

Introduction and Terminology

Fine-tuning adapts a pretrained model to a task or domain through further training. For a language model, supervised fine-tuning (SFT) teaches it to predict supplied responses to prompts. Training on mathematical solutions may improve mathematical performance, but the resulting parameter changes can also alter how the model responds to unrelated questions.

We call improvement in the target domain **adaptation**, and preservation of existing capabilities **retention**. Substantial degradation of previously acquired capabilities is called **catastrophic forgetting**. The challenge is to support adaptation while maintaining retention; whether a particular training procedure causes degradation is an empirical question.

In class, we described fine-tuning through three choices: the training objective, a penalty on departures from the starting model, and restrictions on allowable updates:

$$\min_{\theta} \ \mathcal{L}_{\mathrm{SFT}}(\theta) + \lambda \Omega(\theta, \theta_0), \qquad \theta - \theta_0 \in \mathcal{S}.$$

Here θ_0 denotes the starting parameters, $\mathcal{L}_{\mathrm{SFT}}$ is the supervised response-prediction loss, Ω is a proximity penalty weighted by λ, and $\mathcal{S}$ specifies the allowed updates. The following methods make different choices within this framework.

Full fine-tuning (Full FT) updates all original model parameters. It provides the most freedom to adapt, with no explicit requirement to stay close to the starting model. **L2-SP** adds L2 regularization toward the starting point:

$$\Omega(\theta, \theta_0) = \frac{1}{2} \sum_{j=1}^{P} (\theta_j - \theta_{0,j})^2.$$

Full FT + L2-SP can still update every parameter, but larger departures incur a greater penalty. Unlike ordinary weight decay, its reference is the starting model rather than zero.

Low-Rank Adaptation (LoRA) instead freezes the original model, or backbone, and trains small pairs of matrices called adapters. For a selected weight $W_0 \in \mathbb{R}^{d_{\mathrm{out}} \times d_{\mathrm{in}}}$, the effective weight becomes

$$W = W_0 + \frac{\alpha}{r} BA, \qquad A \in \mathbb{R}^{r \times d_{\mathrm{in}}}, \qquad B \in \mathbb{R}^{d_{\mathrm{out}} \times r}.$$

The adapter rank r limits the correction's rank, and α/r scales it. A maps inputs to r intermediate features; B maps these features to the output space. Higher rank allows richer corrections and increases trainable parameters. Although the backbone remains frozen, the added correction changes the model's behavior and may affect unrelated tasks.

Orthogonal Projection LoRA (OPLoRA) further restricts the adapter correction. It uses the starting weight's leading singular vectors to project corrections outside its dominant input and output subspaces. The protection rank k specifies how many leading singular directions are protected; it is distinct from adapter rank r. This aims to reduce interference with existing transformations, although protecting these directions does not guarantee preserved capabilities.

In this task, you will fine-tune a language model on mathematics using Full FT, Full FT + L2-SP, LoRA at ranks 4 and 16, and OPLoRA at rank 16. No old-task training data or replay may be used. Evaluate mathematical gains alongside changes on other tasks: a method that retains more but learns less may simply adapt weakly, whereas similar mathematical gains with smaller retention losses indicate a more useful trade-off.

Readings

Required:

- Xiong and Xie, *OPLoRA*: Sections 3.2–3.3 and Proposition 2.

Optional:

- Hu et al., *LoRA: Low-Rank Adaptation of Large Language Models*: parameterization, initialization, and merging.
- Li et al., *Explicit Inductive Bias for Transfer Learning with Convolutional Networks*: L2-SP regularization.
- *LoRA Learns Less and Forgets Less* and the *lm-evaluation-harness* documentation.

Dataset, Models, and Experimental Setup

Model. Use the post-trained `Qwen/Qwen3-0.6B`, with thinking disabled during training and generation (`enable_thinking=False`). Start every run independently from this checkpoint. Use seed 6304 and the same examples, order, tokenization, and evaluation prompts throughout. Evaluate the unchanged model once as a shared baseline.

Training data. Select the first **40,000 eligible examples** from `MetaMathQA`, in source order. Exclude examples with empty questions or responses, formatted length above 2,048 tokens, or a `query`/`original_question` matching a GSM8K test question after text normalization. **Do not truncate answers or shuffle the retained examples.**

Format each example as a user question and its supplied assistant response. Compute next-token cross-entropy only on assistant-response tokens and the end-of-turn token; mask prompts and padding. Normalize the accumulated loss over all valid response tokens in the effective batch.

Shared training settings. Train each of the five variants for one epoch:

Setting	Value
Training budget	40,000 examples; 1,250 optimizer updates
Batch size	Microbatch 2; gradient accumulation 16; effective batch 32
Optimizer	AdamW; betas $(0.9, 0.999)$; epsilon 10^{-8}
Weight decay	0.01; exclude biases and parameters with fewer than two dimensions
Learning-rate schedule	Cosine decay; 38 warmup updates
Gradient clipping	Global norm 1.0
Sequence length	Maximum 2,048 tokens
Gradient checkpointing	Enabled

Use FP16 computation without quantization, automatic mixed precision, and dynamic gradient scaling. Full FT variants use FP32 master parameters; adapter variants use a frozen FP16 backbone and FP32 adapters. Keep optimizer states in FP32, unscale before clipping, and log any skipped updates.

Evaluation datasets. Use `lm-evaluation-harness` with fixed task definitions and complete splits. All evaluations are zero-shot.

GSM8K measures mathematical transfer. HellaSwag probes commonsense completion, BoolQ yes/no reading comprehension, and ARC-Easy elementary science questions. Their separate results show whether changes extend beyond mathematics.

Dataset	Split / examples	Method	Primary metric
GSM8K	Test / 1,319	`gsm8k_cot`; generation	Flexible-extraction exact match
HellaSwag	Validation / 10,042	Candidate likelihood	Normalized accuracy
BoolQ	Validation / 3,270	Candidate likelihood	Accuracy
ARC-Easy	Test / 2,376	Candidate likelihood	Normalized accuracy

Evaluation settings. Evaluate all models in FP16 with dropout disabled. For GSM8K, set `num_fewshot=0`, use the non-thinking chat template, greedy decoding, and a 1,024-new-token cap. Use the task's stock stopping rules and answer filters; unextractable answers count as incorrect. For the other benchmarks, use stock completion prompts without a chat template. Report raw accuracy alongside normalized accuracy where available.

Likelihood scoring avoids requiring answers in a particular generated format. GSM8K generation additionally reveals whether fine-tuning changes solution style or answer extractability.

Diagnostic and qualitative inputs. Form an activation-probe set by sampling 32 GSM8K test questions and 32 HellaSwag validation contexts using seed 6304, before inspecting model results. Save the example IDs and reuse this set for every model. For qualitative evaluation, use all 24 prompts from `first_plot_questions.yaml`: eight questions in plain, JSON, and template formats. These prompts probe changes in instruction following, coherence, answer style, and inappropriate content on questions unrelated to mathematics.

Checkpoint and evaluation schedule. Save recovery checkpoints after 9,984, 20,000, 29,984, and 40,000 examples (updates 312, 625, 937, and 1,250), using complete optimizer updates at approximately quarter-epoch intervals. Preserve optimizer, scheduler, gradient-scaler, RNG, and data-position state for resuming.

Stage	Examples seen	Updates	Models evaluated
Baseline	0	0	Unchanged starting model
Midpoint	20,000	625	Full FT only
Final	40,000	1,250	All five variants

Evaluate the shared baseline once, Full FT at midpoint and final, and Full FT + L2-SP, LoRA-4, LoRA-16, and OPLoRA only at final. This gives **seven evaluation conditions**, used for both quantitative benchmarks and qualitative comparisons. The Full FT midpoint shows how its mathematical gains and retention changes develop during training. The first- and third-quarter checkpoints are for recovery only.

Steps

1. **Full Fine-Tuning.** Update all original model parameters using the shared training recipe, a peak learning rate of 2×10^{-5}, and $\lambda = 0$. This run measures mathematical adaptation and retention when all parameters are free to change.

   Logging for all methods. Collect the following for Full FT and **every subsequent training run**, using the checkpoint and evaluation schedule above:

   - SFT loss and learning rate against optimizer update, to plot learning curves.
   - Training time, peak GPU memory, trainable parameter count, and checkpoint sizes, to compare resource use. Include method-specific overhead where applicable.
   - Aggregate benchmark scores and per-example outputs at required evaluation stages, including extracted GSM8K answers and truncation flags. Use these to compute changes from the shared baseline in percentage points.
   - Saved model or adapter states needed for the weight-space, activation, and qualitative analyses below.

   For Full FT, also record full-model displacement $\|\theta - \theta_0\|_2$ at midpoint and final; compare the final value with the regularized run in the next step.

2. **Full Fine-Tuning with L2-SP.** Start again from the original checkpoint and repeat Full FT with the same settings, adding the starting-point penalty with $\lambda = 10^{-3}$. This comparison tests whether the penalty preserves other capabilities without sacrificing mathematical gains.

   Sum over all unique parameters, including biases and normalization parameters, counting tied parameters once. Keep θ_0 fixed. Do not average Ω over parameters.

   In addition to the common logs, record Ω, $\lambda\Omega$, and total loss throughout training, and full-model displacement at final. Track reference-storage and transfer overhead. The other methods use $\lambda = 0$.

3. **LoRA at Ranks 4 and 16.** Train two independent runs with $(r, \alpha) = (4, 8)$ and $(16, 32)$, each using peak learning rate 2×10^{-4}. Freeze the backbone and use dropout 0.05, random A, zero B, and no trainable biases. Target `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, and `down_proj` in every transformer layer.

   Increasing r from 4 to 16 quadruples adapter parameters: investigate whether the extra capacity improves mathematics and changes retention. Record the total adapter parameter count for each run.

4. **OPLoRA.** Train with $r = 16$, $\alpha = 32$, and protection rank $k = 128$. Use the same learning rate, adapter settings, targets, and initial unprojected factors as LoRA-16. For each starting weight $W_0 = U\Sigma V^\top$, freeze its leading singular bases U_k, V_k and implement

$$\widetilde{B} = B - U_k(U_k^\top B), \qquad \widetilde{A} = A - (AV_k)V_k^\top, \qquad \Delta W = \frac{\alpha}{r}\widetilde{B}\widetilde{A}.$$

   Compute the bases once in FP32. Use effective factors rather than dense projection matrices, and export the projected factors for inference. Record basis storage, preprocessing time, and export overhead alongside the common logs. Compare with LoRA-16 to test orthogonal protection at matched adapter rank and initialization.

5. **Evaluate Mathematical Performance and Retention.** Evaluate the required checkpoints using the datasets and fixed scoring settings defined above. Merge adapters into fresh backbone copies, using projected factors for OPLoRA. Check merged versus unmerged logits on a small batch. Save per-example outputs, extracted GSM8K answers, truncation flags, and aggregate scores.

6. **Analyze the Trade-Offs.**

   Benchmark comparisons. Plot training losses for all runs and benchmark scores at the required stages. Report changes from baseline in percentage points. Use final checkpoints for three controlled comparisons: LoRA-4 versus LoRA-16 for rank, Full FT versus Full FT + L2-SP for explicit regularization, and LoRA-16 versus OPLoRA-16 for orthogonal protection. Additionally compare Full FT and LoRA-16 to assess whether similar mathematical gains are accompanied by different retention outcomes. Use the Full FT midpoint to examine its progression during training.

   Weight-space diagnostics. For all five variants at final, and Full FT at midpoint, examine `down_proj` in layers 0, 13, and 27 (zero-based). Using the same starting-weight bases, compute

$$\rho_k = \frac{\|U_k U_k^\top \Delta W\|_F^2}{\|\Delta W\|_F^2}, \qquad \eta = \frac{\|\Delta W\|_F}{\|W_0\|_F}.$$

   The subspace-interference ratio ρ_k measures the fraction of a nonzero update's squared Frobenius norm lying in the protected output subspace spanned by U_k. Intuitively, a high value means the correction is concentrated along the starting weight's strongest output directions, while a value near zero means it acts mainly outside those directions.

   The relative update magnitude η is the ratio of the correction's Frobenius norm to the starting weight's Frobenius norm, regardless of its direction. Intuitively, $\eta = 0.1$ means adding a correction one-tenth the size of the original matrix, whereas $\eta \approx 1$ means the correction is comparable in size to the matrix itself.

   Activation diagnostics. Using your probe set and the benchmark prompt formats, implement forward hooks to cache the starting model's `down_proj` inputs z at the last non-padding prompt token, before any answer continuation. For the same checkpoints as the weight-space diagnostics, apply each local update to the same cached inputs and compare

$$R(z) = \frac{\|\Delta W z\|_2}{\|W_0 z\|_2 + 10^{-8}}$$

   on math and non-math prompts. Use FP32 for these diagnostics. For Full FT variants, $\Delta W = W_{\text{trained}} - W_0$.

   This probes whether math-trained corrections also respond to unrelated inputs. Reusing the same cached activations isolates the local weight change; it does not capture changes elsewhere in the network.

7. **Compare Qualitative Behavior.** Using the qualitative prompts defined above, generate one greedy response per prompt with thinking disabled and a 512-new-token cap for each of the seven evaluation conditions: baseline, Full FT midpoint, and the five final models. Save all 168 responses grouped by question and format. Compare final models with the baseline and with one another; compare Full FT midpoint and final to examine changes during training.

   Annotate the first and last underlying questions in the file's plain-question order, including all three formats and all seven conditions (42 responses). Compare format compliance, coherence/relevance, repetition, unnecessary mathematical framing, and inappropriate content. Flag truncation and support observations with short excerpts. Summarize representative comparisons in the report.

   These comparisons reveal behavioral changes that aggregate scores can miss, including whether mathematical fine-tuning intrudes into unrelated conversations or weakens structured response formatting.

What to Watch For

- Keep training examples, prompt formatting, and scoring settings consistent across runs. Distinguish changes in GSM8K answer extraction or truncation from changes in answer correctness.
- Compare SFT loss across methods separately from the L2-SP total objective. The penalty is anchored to the starting parameters; AdamW weight decay has a different reference.
- Equal examples and optimizer updates do not imply equal compute. Account for different learning rates and method-specific overhead when interpreting comparisons.
- Use OPLoRA's projected factors for diagnostics and inference. Separate numerical verification of its constraints from evidence about benchmark performance; local activation measurements do not establish a cause of behavioral changes.

Required Evidence

- **Benchmark results:** one table covering all seven evaluation conditions and all four benchmarks, with primary scores and changes from baseline in percentage points. Use it to discuss Full FT's progression and the final-model comparisons.
- **Learning and regularization:** a combined SFT-loss plot for the five runs, with a concise comparison of Full FT and L2-SP penalty strength and final parameter displacement.
- **Resources:** a compact comparison of trainable parameters, training time, peak GPU memory, and checkpoint sizes for the five runs.
- **Diagnostics:** a compact figure or table summarizing subspace interference, relative update magnitude, and math/non-math activation corrections. Explain how these measurements relate to the benchmark results and what they cannot establish.
- **Qualitative findings:** a brief summary of the fixed 42-response annotation subset and a small number of aligned excerpts comparing the same question and format across models. Explain the selection of illustrative examples and any observed changes.

Research Questions

1. **Specialization and retention.** How does mathematical fine-tuning affect the balance between mathematical performance and other capabilities, and how does this balance vary across methods and training stages?
2. **Restrictions on adaptation.** How do proximity regularization, adapter rank, and orthogonal protection affect the trade-off between adaptation and retention?
3. **Parameter changes and behavior.** What relationships exist between the magnitude and geometry of parameter updates, their activation on different inputs, and changes in model behavior?
4. **Performance and computational cost.** How do the methods compare in the balance they achieve between mathematical performance, retention, and computational cost?
5. **Behavior beyond benchmark scores.** How does mathematical fine-tuning affect responses to unrelated questions, and to what extent do benchmark scores reflect these behavioral changes?

2 Vision-Language Models and Visual Grounding

Introduction and Terminology

A **vision-language model (VLM)** combines image representations with a language model to generate image-conditioned responses. Connecting two pretrained models requires an interface between their representation spaces and training that teaches the language model to use the visual information.

A **vision encoder** converts an image into patch features. A trainable **connector** maps these features into the language model's embedding space, producing continuous **visual tokens**. These tokens are inserted alongside text embeddings, allowing the language model to attend to image information when generating its response. Here, a two-layer MLP connects a frozen SigLIP encoder to Qwen:

$$F(I) \in \mathbb{R}^{196\times 768}, \qquad Z(I) = g_\phi(F(I)) \in \mathbb{R}^{196\times 1024},$$

where I is the image and ϕ denotes connector parameters.

Caption alignment trains the connector to make visual features useful for predicting image descriptions. **Visual instruction tuning** trains the system to answer image-conditioned questions. LoRA adapters allow the language model to adapt while its original parameters remain frozen. Training first on captions, then simple visual questions, and finally natural VQA forms a **curriculum**. **Replay** mixes examples from earlier stages into later training.

Visual grounding means that an answer is supported by image evidence. A model can instead exploit **language priors**, such as frequently occurring answers to a question type. A **complementary pair** contains the same question

applied to two images with different reference answers. Question wording alone cannot identify both answers. Changing the image tests whether predictions respond to visual evidence; correctness determines whether that response is useful.

In this task, you will build and train a SigLIP–Qwen VLM through three sequential stages, then investigate grounding using complementary pairs and image controls. An optional experiment compares 196 and 49 visual tokens.

Readings

Required:

- Liu et al., *Visual Instruction Tuning*: visual connectors, feature alignment, and instruction tuning.

Optional:

- Goyal et al., *Making the V in VQA Matter*: complementary pairs and language priors.
- *Sigmoid Loss for Language Image Pre-Training* and the VQA answer-normalization and scoring code.

Dataset, Models, and Experimental Setup

Models. Start from the original post-trained Qwen checkpoint. Disable thinking during training and generation (`enable_thinking=False`).

Component	Model / architecture	Training policy
Vision encoder	`google/siglip-base-patch16-224`	Always frozen
Connector	Linear 768 → 1024, GELU, Linear 1024 → 1024	Trainable throughout
Language model	`Qwen/Qwen3-0.6B`	Original weights frozen; LoRA in Stages 2–3

Use the 196 final patch features in their 14×14 spatial order, applying SigLIP's final layer normalization exactly once. Cache them in FP16 with deterministic image preprocessing, then remove the vision encoder from GPU memory before training Qwen.

Data. Prepare three components using COCO 2014 captions and instance annotations and VQAv2 complementary pairs. A question pair contains the *same question* applied to two different images with different reference answers; each pair therefore provides two question–image examples.

- **COCO captions:** select existing image–caption examples for caption alignment and later replay. Use three captions per selected training image and one per development image.
- **Annotation-derived grounding:** write a script to generate template questions and answers from COCO object annotations. For *presence*, ask "Does this image contain [object]?" and pair an image with an annotated object with an image without annotations for that category, giving yes/no targets. For *counting*, ask "How many [objects] can you see?" and pair images with different annotated counts between 1 and 4. These questions are generated programmatically rather than manually written.
- **Natural VQAv2:** select existing complementary pairs, questions, and human answers. Keep pairs whose normalized consensus answers differ and have agreement from at least 8 of 10 annotators for each image. Use an original answer string from the consensus group as the training target; retain all ten human answers for scoring. Apply the same answer normalization throughout.

Splits and sizes. Use `train2014` images for training and `val2014` images for development and final test. Form pairs within each split. Keep all development and final-test sets image-disjoint from one another and from training. Training images may recur across curriculum stages; evaluation images cannot enter training or replay.

Dataset	Training	Development	Final test
COCO captions	6,000 images × 3 captions = 18,000 examples	96 images, one caption each	—
Annotation-derived grounding	Generated pair pools, sampled during training	24 pairs / 48 questions	64 pairs / 128 questions
Natural VQAv2	1,536 pairs / 3,072 questions	32 pairs / 64 questions	96 pairs / 192 questions

The annotation-derived counts combine presence and counting pairs. No fixed number of unique training pairs is required: construct reusable pools, then draw from them according to the stage sampling probabilities in the Steps subsection. The update budgets determine the amount of training; pairs may be sampled repeatedly.

For natural VQAv2, the yes/no, number, and other pair counts are **640/256/640** for training, **12/8/12** for development, and **36/24/36** for final test. Use seed 6304 for data selection and save the selected image, question, and pair IDs.

Shared training settings. Continue the same model through all three stages.

Setting	Value
Seed	6304
Batch size	Microbatch 8; accumulation 2; effective batch 16
Optimizer / weight decay	AdamW / 0.01
Gradient clipping	Global norm 1.0
Learning-rate schedule	Per-stage 5% warmup, then cosine decay to 10% of peak LR
Gradient checkpointing / quantization	Enabled / none
Content-token caps	Question 80; caption target 64; short-answer target 16

Use FP16 autocast with dynamic gradient scaling, a frozen FP16 backbone, and FP32 trainable parameters and optimizer states. Unscale before clipping.

Compute causal cross-entropy on caption/answer tokens and the end-of-turn token; mask visual tokens, prompts, and padding. Average supervised-token loss within each example, then across the effective batch:

$$\mathcal{L} = -\frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \frac{1}{|\mathcal{R}_i|} \sum_{t \in \mathcal{R}_i} \log p(y_{i,t} \mid I_i, q_i, y_{i,<t}).$$

This example-weighted loss gives each caption or short answer equal weight.

Training budget and checkpoints. The main run has 4,500 updates and 72,000 example presentations, including replay.

Stage	Updates	Connector peak LR	LoRA peak LR
1. Caption alignment	2,250	10^{-3}	—
2. Visual instruction tuning	1,500	10^{-4}	2×10^{-5}
3. Natural VQA specialization	750	5×10^{-5}	10^{-5}

Save resumable checkpoints every 250 main-run updates and connector/adapter weights after each stage. Include optimizer, scheduler, scaler, sampler position, stage progress, and RNG state. The completed Stage 3 model is the final model.

Development evaluation. Evaluate before training, every 500 main-run updates, and at stage endpoints 2,250, 3,750, and 4,500. Track caption reference loss and reference-answer negative log-likelihood (NLL) on both VQA development suites under correct, complementary, and no-image inputs. For answer NLL, exclude termination and average token NLL within each example before averaging examples. Define the image-swap penalty

$$\Delta_{\text{NLL}} = \text{NLL}(y \mid q, I_{\text{partner}}) - \text{NLL}(y \mid q, I_{\text{correct}}).$$

Positive values mean the correct image makes the original reference more likely.

Final evaluation. Evaluate both final-test suites using greedy decoding, thinking disabled, a 16-new-token cap, and batch size 8. Keep the question and original references fixed across four conditions:

Condition	Image input
Correct	Original image
Complementary swap	Partner image with a different answer
Mismatched	Unrelated image from another pair in the same suite
No image	Visual tokens omitted; text instructions unchanged

Fix the mismatched-image mapping before generation, excluding both images of the original pair, and reuse it across comparisons. For no-image inputs, remove visual tokens and rebuild masks and positions.

Score grounding questions with normalized exact match, separating presence and counting. Score natural VQAv2 with local VQA soft consensus accuracy against all ten answers, separating yes/no, number, and other. For normalized prediction $\hat{a}$ and references $a_1, \ldots, a_{10}$, use

$$s(\hat{a}) = \frac{1}{10} \sum_{i=1}^{10} \min\left(1, \frac{1}{3} \sum_{j \neq i} \mathbf{1}[\hat{a} = a_j]\right).$$

These are local subset results, not full-benchmark server scores.

For each suite and image condition, also report **pair accuracy** (both predictions match their original canonical answers), **identical-answer rate**, **empty-answer rate**, mean reference-answer NLL, and generation time per answer. Keep empty outputs in accuracy denominators. Report correct-image advantages over complementary swaps in accuracy and NLL. Give 95% confidence intervals using 4,000 bootstrap resamples of complete pairs, matching sampled pair IDs across conditions when computing differences.

Steps

1. **Build the Visual Interface.** Implement the connector and insert its visual embeddings into the user message before the question, preserving Qwen's chat structure. Construct matching labels, attention masks, and position indices. Check feature shapes, frozen parameter sets, connector gradients, and split isolation. Calibrate connector output magnitude against text embeddings using training data only, and record the rule and resulting scale.

   Logging throughout training. Save training loss, mixture counts, development metrics, and configurations. Record trainable parameters by stage, runtime, and peak GPU memory, separating feature caching from the training run. Save raw references, generations, and per-example metrics for evaluation.

2. **Caption Alignment.** Train only the connector for two passes over the 18,000 caption examples (2,250 updates). This stage learns an interface between frozen visual features and the language model before adding question-answer training.

3. **Visual Instruction Tuning.** Continue for 1,500 updates. For each draw, randomly choose presence with probability 50%, counting with probability 25%, or caption replay with probability 25%. Train the connector and newly enabled LoRA adapters, keeping original backbone parameters frozen. Use LoRA rank 16, alpha 32, dropout 0.05, no trainable biases, and targets `q_proj`, `k_proj`, `v_proj`, and `o_proj`. This stage introduces explicit visual questions while retaining caption examples.

   Each draw contributes two examples: a complete question pair, or two caption examples for replay. This keeps the sampling probabilities comparable across components while preserving pairs. Use the seeded random sampler for both question-training stages; do not enforce exact batch or stage quotas. Repeated sampling of training pools is allowed.

4. **Natural VQA Specialization.** Continue the same connector and adapters for 750 updates. For each draw, randomly choose natural VQAv2 with probability 70%, presence replay with probability 10%, counting replay with probability 10%, or caption replay with probability 10%. This stage moves from annotation-derived questions to natural questions and answers.

5. **Evaluate Visual Grounding.** Apply the development and final evaluation protocols above. Compare correct-image results with complementary, mismatched, and no-image controls, reporting the two suites separately. Examine whether changes under complementary swaps follow the partner image's answer or simply produce an unsupported answer.

6. **Analyze Training and Model Behavior.** Plot development metrics across stages alongside training loss. Compare reference likelihood, generated accuracy, and pair-level results across image conditions and question types. Present complementary images with the shared question, references, and predictions, using a stated selection rule for illustrative examples. Discuss image use, answer consistency, and the limitations of the observed results.

7. **Optional Visual-Token Compression.** Repeat the complete recipe with 49 visual tokens, averaging each spatial 2×2 block of the 14×14 feature grid before the connector. Initialize a new connector and adapters, keeping seed, data, training budgets, and evaluation unchanged. Compare grounding, accuracy, runtime, and memory with the 196-token run. This experiment is optional.

What to Watch For

- Freezing Qwen's parameters must still allow gradients to reach the connector. Disabling autograd for its entire forward pass prevents connector learning.
- COCO annotations can omit objects or contain ambiguous counts. Inspect labels independently of model predictions; keep evaluation images out of training and replay.
- Swapped-image scores use the original references. Image sensitivity, correctness under the original image, and following the new image are different measurements.
- No-image inputs also shorten the prefix. Complementary swaps preserve question wording and visual-token count.
- Teacher-forced answer likelihood and free generation measure different behavior.
- Bootstrap complete pairs, preserving their shared question structure. Do not select checkpoints using final-test results.

Required Evidence

- **Learning curves:** training loss, development caption loss, answer NLL, and image-swap penalties, with stage boundaries identified.
- **Final results:** compact tables for both suites under all four image conditions, including question-type results, pair diagnostics, reference NLL, correct-image advantages, and pair-bootstrap intervals.
- **Qualitative findings:** aligned complementary-image examples showing the question, references, and model responses, with an explanation of example selection and observed behavior.
- **Resources:** training and generation time, peak GPU memory, and trainable parameters by stage, including feature-caching costs. Optional compression results may be reported separately.

Research Questions

1. **Learning a visual interface.** How does the training curriculum affect the model's ability to describe images and answer visual questions? What carries over or changes between stages?
2. **Grounding and generalization.** To what extent do answers depend correctly on visual evidence, and how does this vary between annotation-derived and natural questions?
3. **Measuring image use.** How consistent are the conclusions suggested by reference likelihood, generated answers, pair-level metrics, and qualitative behavior?
4. **Capabilities and cost.** What capabilities and limitations characterize this VLM, and what computational costs accompany its training and inference?

3 Test-Time Compute: Sampling, Selection, and Adaptive Inference

Introduction and Terminology

Test-time compute is computation spent answering a query while model parameters remain fixed. Additional computation can produce more candidate solutions, develop an existing solution further, or evaluate alternatives before choosing an answer. The question is how effectively this computation is used, rather than simply how many tokens are generated.

Parallel sampling generates independent reasoning trajectories, or **rollouts**, for the same query. **Best-of-N selection** returns the highest-scoring candidate, whereas **self-consistency** aggregates candidates by their final answers. Repeated sampling can increase the chance of finding a correct answer, but a selection rule must recognize it. **Oracle coverage** measures whether any candidate is correct; it is an analysis upper bound that uses reference answers, not a deployable selection method.

Sequential refinement conditions a new response on an earlier solution and a request to review it. It spends computation on reconsideration rather than independent exploration. Search methods combine branching with intermediate evaluation; an **outcome verifier** evaluates completed solutions, while a **process verifier** evaluates intermediate steps. Here, model confidence and visual engagement provide internal quality signals without an additional verifier model.

Self-certainty summarizes the model's full next-token distributions, while sampled-token log-probability measures support for the tokens actually generated. For VLMs, **visual engagement** measures the share of attention allocated to the image. **VisConf** combines visual engagement with candidate-calibrated self-certainty. **Rank-weighted consensus (RWC)** combines rollout quality with agreement by assigning larger votes to higher-ranked rollouts.

Adaptive sampling varies the number of rollouts according to the evidence observed so far. A stopping policy trades the cost of another sample against its potential to change or improve the answer. This task investigates these choices using supplied LLM/VLM rollouts, a small live refinement experiment, and simulated adaptive inference.

Readings

Required:

- Kang et al., *Scalable Best-of-N Selection for Large Language Models via Self-Certainty*: distributional confidence.
- Usman et al., *VisConf: Quality-Aware Consensus for Best-of-N in Vision-Language Model Reasoning*: Sections 3–5, including calibration, fusion, and RWC.

Optional:

- Wang et al., *Self-Consistency Improves Chain of Thought Reasoning in Language Models*: sampling and answer aggregation.
- Madaan et al., *Self-Refine: Iterative Refinement with Self-Feedback*: sequential feedback and revision.
- *Adaptive-Consistency*, *Tree of Thoughts*, and *Let's Verify Step by Step*.

Dataset, Models, and Experimental Setup

Supplied rollout data. The assignment supplies `ATML-PA3-Data`, containing 32 sampled rollouts per question for each model–dataset combination below. Use these generations for the offline experiments; only the small MATH experiment in Step 4 requires new generation.

Model	Type	Dataset	Questions	Rollouts
Qwen2.5-3B-Instruct	LLM	MATH test subset	1,000	32,000
Qwen2.5-VL-3B-Instruct	VLM	MathVista testmini	1,000	32,000

Both exports use temperature 0.7, top-p 0.8, top-k 20, repetition penalty 1.0, and a 2,048-new-token cap. Read the supplied `README.MD` for field definitions and generation details.

Each model folder contains `generations.parquet` and `probability_metrics.parquet`; the VLM folder also contains `attention_metrics.parquet`. Join on `(dataset, sample_id, rollout_id)`. The files supply text, token IDs, lengths, extracted answers, gold answers, sampled-token `logp`, and full-distribution `kl_u_p`; VLM attention is provided separately for image, prompt-text, and generated-text groups. Token measurements describe the prediction of the corresponding generated token, and termination tokens are excluded. Images are not needed for offline analysis.

Development and evaluation splits. Within each dataset, sort the 1,000 question IDs lexicographically and permute them with NumPy's `default_rng(6304)`. Use the first 200 questions for development and the remaining 800 for held-out evaluation. Keep all rollouts of a question together. Use the first 50 MATH evaluation questions in this permutation for the live generation subtask. Save the split IDs and use seed 6304 for your analysis and new generation.

Answers and correctness. Use the supplied extracted answers for stored rollouts. Normalize equivalent answers before voting: use `math-verify` for MATH mathematical equivalence, and MathVista's answer-type, choice, and precision rules using the released question metadata. Represent multiple-choice answers as option letters. Form answer groups without consulting gold answers, and use the same equivalence rules for grading. Fix the scorer before final

evaluation and report its version. Map missing extractions to one `NO_ANSWER` group, always graded incorrect; keep them in voting, cost, and accuracy accounting. Correctness labels are not supplied and must be computed for analysis.

Offline evaluation. Use candidate budgets $N \in \{1, 2, 4, 8, 16, 32\}$. Generate ten seeded random permutations of the 32 rollout IDs per question and use nested prefixes of each permutation. Reuse these orders across methods and stopping policies. Average outcomes over orders within a question, then across questions. Resolve score or answer-vote ties by the earliest supporting rollout in the current order; use average ranks for tied RWC scores. At $N = 1$, every rule returns the only candidate.

Report answer accuracy, oracle coverage, and **generated-token cost**: the sum of retained token counts for all candidates consumed by the method. FirstAns consumes only its first candidate. Token counts are an offline cost proxy; they do not include prefill, attention collection, or scoring overhead. Use 1,000 paired question-level bootstrap resamples for 95% intervals on main accuracy differences, after averaging over orders.

Live-generation settings. Use the original `Qwen/Qwen2.5-3B-Instruct` revision listed in the supplied `README.MD`, with FP16 weights, no quantization, and batch size 1. Use temperature 0.7, top-p 0.8, top-k 20, repetition penalty 1.0, and the token limits in Step 4. Use its native chat template, the system message "You are a helpful assistant.", and the original problem followed by:

> Think step by step inside `<think> </think>` tags, then give your final answer inside `\boxed{}`.

Extract the last boxed answer using the balanced-brace rule in `README.MD`. Keep model weights fixed and save all prompts, responses, seeds, lengths, and truncation flags. Measure end-to-end inference time on the same GPU, excluding model loading; also record input/output tokens and peak GPU memory. These FP16 runs are a new controlled comparison, not a reproduction of the supplied samples.

Steps

1. **Establish the Opportunity from Repeated Sampling.** Load and join the supplied tables, verify token-list alignment, and prepare answer groups and correctness labels. For each budget $N \in \{1, 2, 4, 8, 16, 32\}$, measure FirstAns accuracy, oracle coverage, and the number of distinct answer groups.

   Decompose selection failures into questions where no correct candidate was generated and questions where a correct candidate was available but missed. This separates limitations of sampling from limitations of selection. Save per-question results, candidate IDs, selected answers, and token costs throughout the task.

2. **Implement Quality Scoring and Answer Aggregation.** For rollout i with T_i tokens, compute mean log-probability and self-certainty:

$$L_i = \frac{1}{T_i} \sum_t \log p_{i,t}(y_{i,t}), \qquad K_i = \frac{1}{T_i} \sum_t k_{i,t}, \qquad k_{i,t} = D_{\mathrm{KL}}(U \| p_{i,t}).$$

   Here U is uniform over the vocabulary. Read the token measurements from `logp` and `kl_u_p`; they use raw model distributions before sampling transformations. Larger L_i means greater support for the sampled tokens; larger K_i means greater divergence from uniform, not a calibrated probability of correctness.

   For MathVista, compute visual engagement from the supplied group attention totals:

$$v_{i,t} = \frac{A_{i,t}^{\text{image}}}{A_{i,t}^{\text{image}} + A_{i,t}^{\text{prompt}} + A_{i,t}^{\text{generated}}}, \qquad V_i = \frac{1}{T_i} \sum_t v_{i,t}.$$

   These totals exclude some special positions and need not sum to one; normalize as shown rather than using raw image-attention mass.

   Implement VisConf by pooling token KL values across the *current candidate set* G and calibrating them with its 5th and 95th percentiles:

$$c_{i,t} = \mathrm{clip}_{[0,1]} \left(\frac{k_{i,t} - Q_{.05}(G)}{\max(Q_{.95}(G) - Q_{.05}(G), 10^{-8})} \right).$$

   Average calibrated confidence within each rollout and multiply it by visual engagement:

$$C_i = \frac{1}{T_i} \sum_t c_{i,t}, \qquad Z_i = V_i C_i.$$

Use linear-interpolation percentiles. Recompute calibration for each candidate prefix; do not normalize using unseen rollouts.

For a quality score Q_i, implement both highest-score selection and RWC. With descending average rank r_i among $N > 1$ candidates, where rank 1 is best, use

$$w_i = \frac{N - r_i}{N - 1}, \qquad W(a) = \sum_{i:a_i=a} w_i, \qquad \widehat{a} = \arg\max_a W(a).$$

Implement the following comparisons at every candidate budget:

- **Both datasets:** FirstAns, self-consistency (one vote per rollout), highest mean log-probability, highest self-certainty, and self-certainty + RWC.
- **MathVista additionally:** vision-only and VisConf, each with highest-score selection and RWC.

Keeping the score fixed while changing aggregation isolates the value of consensus; keeping aggregation fixed while changing the score isolates the value of the quality signal.

3. **Investigate What the Signals Capture.** At $N = 32$, examine within-question associations between rollout length, scores, and correctness. Compare correctness in each score's top and bottom quartiles within a question, then average the differences across questions. Include shortest-rollout selection as a length-only control.

   For MathVista, compute confidence–vision correlation within each rollout and summarize these correlations across questions. Compare VisConf with tokenwise fusion

$$Z_i^{\text{token}} = \frac{1}{T_i} \sum_t c_{i,t} v_{i,t}, \qquad Z_i^{\text{token}} - Z_i = \text{Cov}_t(c_{i,t}, v_{i,t}).$$

   Evaluate this ablation under the same two aggregation rules. It tests whether combining signals at each token is equivalent to combining their rollout summaries. Use a fixed set of illustrative trajectories to examine where signals agree, disagree, or vary over generation.

4. **Compare Parallel Sampling with Sequential Refinement.** On the fixed 50-question MATH subset, run the following strategies with the same **3,072-token total output allowance** per question:

Strategy	Generation and final answer
Single longer response	One response capped at 3,072 new tokens
Parallel sampling	Three independent responses capped at 1,024 tokens each; self-consistency selects the answer
Sequential refinement	Initial response capped at 1,024 tokens, then one critique-and-revision response capped at 2,048; return the revised answer

   Reuse the first parallel response as the initial response for refinement. Append it as an assistant message, followed by this user instruction:

   > Review the solution above. Identify and correct any errors; if it is correct, retain its answer. Give your final answer inside `\boxed{}`.

   Generate critique and revision together in that second response, without gold answers or correctness feedback. Use independently seeded calls and keep the generation instructions fixed across questions.

   Early termination may leave part of an allowance unused, so report actual tokens and runtime alongside accuracy and extraction/truncation rates. Charge the shared initial response to both strategies when comparing their costs. Count all critique tokens and the longer revision prompt. Report wrong-to-correct and correct-to-wrong transitions relative to the shared initial answer; do not use gold answers to choose between initial and revised outputs.

5. **Implement Adaptive Sampling.** Return to the supplied pools and reveal rollouts in the saved random orders. Check stopping after $n \in \{4, 8, 16\}$ candidates; otherwise stop at 32. Both controllers below return the current **self-consistency answer**, so the comparison isolates stopping rather than changing the final selector.

   - **Agreement-based:** stop if there is a unique, non-`NO_ANSWER` leading answer supported by at least a fraction τ of the n candidates.

- **Quality-aware:** require the same agreement condition, plus agreement between the self-consistency and RWC winners and an RWC vote margin of at least γ. Use self-certainty scores for MATH and VisConf for MathVista.

Define the RWC margin as the difference between the largest and second-largest answer votes divided by total vote weight; use zero for the runner-up vote when only one answer group exists. Recompute scores, VisConf calibration, and ranks using only revealed candidates.

On each dataset's development set, consider $\tau \in \{0.5, 0.625, 0.75, 0.875, 1.0\}$ and, for quality-aware stopping, $\gamma \in \{0, 0.1, 0.25, 0.5\}$. Include the policy that always consumes 32 candidates as a fallback. For each controller family, select the setting with the lowest mean token cost whose development accuracy is no more than one percentage point below fixed-32 self-consistency, averaging over the ten orders. Break ties by higher accuracy, then smaller τ and γ.

Freeze the selected settings before held-out evaluation. Report accuracy, mean token cost, mean rollouts, and the stopping-count distribution, alongside fixed-N self-consistency. Analyze incorrect early stops, including cases where fixed-32 selection would have recovered the correct answer. These are offline simulations of stopping between completed rollouts, not within a rollout.

6. **Analyze Compute Allocation and Failure Cases.** Compare accuracy against candidate count and generated-token cost, separating oracle coverage from achieved selection accuracy. Examine whether additional samples add useful alternatives or repeat existing answers, and how score quality and aggregation affect the remaining selection gap.

   Compare adaptive policies with fixed budgets at similar observed costs. Relate the stopping behavior to answer agreement and to the fraction of correct candidates in the full pool, using the latter only for retrospective analysis. Present representative cases of selection disagreement, revision, and premature stopping using a stated example-selection rule.

What to Watch For

- Answer equivalence affects both voting and grading. Missing extractions and capped outputs remain part of the experiment; do not remove them to improve scores.
- Gold answers and future candidates are available for retrospective analysis, not for selection, stopping, or candidate-relative normalization.
- Consensus, self-certainty, and visual attention are evidence signals, not guarantees of correctness. Token-level associations do not establish causal visual grounding.
- Compare live strategies using their actual input/output costs and runtime as well as output allowances. Three short responses can encounter different truncation rates from one long response.
- Treat offline token cost as a proxy. Scoring and collecting all-layer attention have additional deployment costs that are not timed in the supplied export.
- Bootstrap questions, not individual rollouts. Results compare two model–dataset combinations; they do not isolate the causal effect of adding vision.

Required Evidence

- **Sampling and selection:** accuracy and oracle-coverage curves for both datasets, a compact method comparison, and the decomposition of sampling versus selection failures.
- **Signal analysis:** length and correctness associations, confidence–vision relationships, and the tokenwise/trajectory-level fusion comparison.
- **Live inference:** accuracy, actual input/output tokens, runtime, truncation/extraction rates, and revision transitions for the three strategies on the 50 questions.
- **Adaptive inference:** development-selected thresholds, held-out accuracy–cost results, stopping distributions, and comparisons with fixed budgets.
- **Qualitative findings:** a small set of aligned examples illustrating the observed selection, refinement, and stopping behavior.

Research Questions

1. **Exploration and selection.** What limits the benefit of repeated sampling, and how does the balance between candidate coverage and selection quality change with compute?
2. **Evidence and consensus.** How do confidence, visual engagement, and answer agreement contribute to selecting a final answer, and what explains their successes or failures?
3. **Breadth and refinement.** How does spending computation on independent alternatives compare with a longer response or sequential reconsideration?
4. **Adaptive allocation.** When is another rollout useful, and how effectively can the observed evidence guide stopping without access to correctness?
5. **Compute and reliability.** Which strategies achieve useful accuracy–cost trade-offs, and what do the observed failures reveal about their reliability and limitations?

4 Synthesis of Results & Report Writing Guidelines

Present the three tasks in one PDF report, with a separate results and analysis subsection for each task. **A cross-task discussion is not required.** Within each task, use the research questions to organize a connected analysis of your findings rather than answering each question separately.

Suggested Report Organization: Abstract; brief Introduction; Experimental Setup/Methods; Results and Analysis for Tasks 1–3; brief Conclusion; References. Keep method summaries concise and devote most of the report to results, comparisons, and interpretation.

Writing and Presentation:

- **Support conclusions with evidence.** Explain what the comparisons reveal, rather than only restating scores. Discuss unexpected or inconclusive results; agreement with an anticipated outcome is not required.
- **Make comparisons clear.** Identify the baseline, model/checkpoint, dataset split, and metric. Distinguish percentage-point changes from relative percentages, and report compute costs alongside performance where required.
- **Use compact figures and tables.** Label axes, units, and methods clearly. Captions should make the comparison understandable; refer to each figure or table in the analysis. Avoid screenshots of logs or raw notebook output.
- **Prioritize the main findings.** Include the central evidence for each task in the main report. Put detailed breakdowns, additional examples, and implementation details in an appendix, and reference them where relevant. State any incomplete experiments or deviations from the prescribed setup.

Deliverables:

- **GitHub Repository:** code, configurations, split/example IDs, environment specification, machine-readable results, and a top-level README explaining how to reproduce each task. Attribute reused external code. Do not commit raw datasets or unnecessary large checkpoints.
- **PDF Report:** 8 pages of main content in the course NeurIPS format introduced in PA0. References and an optional appendix do not count toward this limit. The appendix supplements the main report; graders are not required to read it. Include your repository link at the end of the abstract and cite the papers, datasets, and tools used.
- **LMS Submission:** upload the PDF and submit an accessible GitHub link in the format announced on LMS.

Before You Submit: Check that evaluation data did not influence training or hyperparameter selection, and that Task 3 selection and stopping rules used neither gold answers nor unrevealed candidates. Every reported number should trace to a saved result file or reproducible command. Follow the AI usage policy stated at the beginning of this assignment.